
Notice that the identities of the lists in `spam` and `eggs` are different. The `eggs = spam[:]` line creates a *shallow copy* of the list in `spam`, whereas `eggs = spam` would copy only the reference to the list. But the `[:]` does look a bit odd, and using the `copy` module's `copy()` function to produce a shallow copy of the list is more readable:

```
>>> # Pythonic Example
>>> import copy
>>> spam = ['cat', 'dog', 'rat', 'eel']
>>> eggs = copy.copy(spam)
>>> id(spam) == id(eggs)
False
```

You should know about this odd syntax in case you come across Python code that uses it, but I don't recommend writing it in your own code. Keep in mind that both `[:]` and `copy.copy()` create shallow copies.

Pythonic Ways to Use Dictionaries

Dictionaries are at the core of many Python programs because of the flexibility that key-value pairs (discussed further in Chapter 7) provide by mapping one piece of data to another. Therefore, it's useful to learn about some of the dictionary idioms Python code commonly uses.

For further information about dictionaries, consult Python programmer Brandon Rhodes's incredible talks about dictionaries and how they work: "The Mighty Dictionary" at PyCon 2010, viewable at <https://invpy.com/mightydictionary>, and "The Dictionary Even Mightier" at PyCon 2017, viewable at <https://invpy.com/dictionaryevenmightier>.

Use `get()` and `setdefault()` with Dictionaries

Trying to access a dictionary key that doesn't exist will result in a `KeyError` error, so programmers will often write unpythonic code to avoid the situation, like this:

```
>>> # Unpythonic Example
>>> numberOfPets = {'dogs': 2}
```
